

A Roofline Study of FlashAttention and 4-bit Quantized LLM Inference on NVIDIA L4

Nathan Gong (ng2695), Jiaxiong Hu (jh4878), Fu Liu (fl2597), and Jason Zeng (jlz2128)

Abstract—Large language model inference is constrained not only by peak compute throughput, but also by memory bandwidth, attention-kernel efficiency, GPU memory capacity, and KV-cache growth. In this project, we perform a roofline-aware performance study of LLM inference using Llama 3.1 8B Instruct on an NVIDIA L4 GPU [3], [4]. We compare four inference configurations: FP16 baseline, FlashAttention only, 4-bit weight quantization only, and FlashAttention combined with 4-bit quantization [1], [2], [5], [6]. We evaluate sequence lengths 128, 256, 512, 1024, and 2048, measuring prefill latency, decode throughput, peak GPU memory, KV-cache footprint, arithmetic intensity, and achieved performance.

Our results show that FlashAttention is the primary performance optimization, especially for long-context prefill and decode throughput. At sequence length 2048, FlashAttention reduces prefill latency from 1668 ms to 1043 ms and improves decode throughput from 10.74 tok/s to 14.10 tok/s. 4-bit quantization, in contrast, provides the clearest memory benefit, reducing peak decode memory from 16.19 GB to 6.32 GB at sequence length 2048. However, quantization does not improve decode throughput in this implementation; quantized decode reaches only 7.15 tok/s at sequence length 2048, likely due to dequantization and kernel overhead. KV-cache memory grows with sequence length and remains identical across configurations because weight-only quantization does not compress runtime key/value activations. Roofline analysis shows that quantization increases arithmetic intensity, while FlashAttention improves realized performance through more efficient attention execution.

I. INTRODUCTION

Transformer inference has two major phases: **prefill** and **decode**. During prefill, the model processes the entire input prompt and constructs the KV cache. During decode, the model generates tokens autoregressively, one token at a time, repeatedly accessing model weights and cached key/value tensors.

These two phases stress the GPU differently. Prefill is more parallel and attention-heavy, while decode is more memory-sensitive because it repeatedly loads weights and KV-cache entries for each generated token. Therefore, optimizations such as FlashAttention and quantization may affect the two phases differently.

This project studies two practical LLM inference optimizations:

- **FlashAttention**, which improves attention execution by reducing memory movement and improving kernel efficiency [1].
- **4-bit weight quantization**, which reduces model parameter memory and memory bandwidth requirements, but may introduce dequantization overhead [2], [5].

Rather than only reporting speedup, we analyze the results through a roofline framework to understand whether performance changes are driven by arithmetic intensity, memory footprint, or kernel efficiency [3].

II. EXPERIMENTAL SETUP

A. Model and Hardware

Experiments use Llama 3.1 8B Instruct on an NVIDIA L4 GPU [4]. For reproducibility, the Hugging Face checkpoint used in the code is meta-llama/Meta-Llama-3.1-8B-Instruct [4].

The roofline analysis uses the following NVIDIA L4 assumptions from our profiling code:

```
FP16 peak throughput: 121 TFLOP/s
Memory bandwidth: 300 GB/s
```

B. Configurations

We evaluate four inference configurations:

TABLE I
INFERENCE CONFIGURATIONS

Configuration	Precision	Attention Backend	Description
Baseline	FP16	SDPA (math)	Standard half-precision inference
FlashAttention	FP16	FlashAttention	Optimized attention backend
Quantized	4-bit	SDPA (math)	Weight-only 4-bit quantization
FA + Quantized	4-bit	FlashAttention	Combined FlashAttention and 4-bit quantization

C. Sequence Lengths and Warmup Protocol

We evaluate five sequence lengths:

128, 256, 512, 1024, 2048

The 128-token run is the first benchmark point for each configuration. Therefore, we treat it as a warmup / cold-start / short-context point. This run may include one-time effects such as CUDA kernel initialization, memory allocator setup, attention backend setup, and GPU clock ramp-up. We still report the 128-token result because short-context inference is practically relevant, but our main steady-state scaling analysis focuses on the 256–2048 token range.

D. Metrics

For each configuration and sequence length, we collect:

- Prefill latency in milliseconds
- Decode throughput in tokens/sec
- Prefill and decode peak GPU memory
- KV-cache memory
- Estimated FLOPs
- Arithmetic intensity
- Achieved performance
- System telemetry through W&B

III. METHODOLOGY

A. Phase-Isolated Benchmarking

The benchmark separates the inference process into prefill and decode. This avoids mixing two fundamentally different workloads into a single throughput number. The prefill phase performs one full forward pass over the prompt and produces the first-token logits and KV cache. The decode phase then generates subsequent tokens using the stored `past_key_values`. This phase isolation is important because prefill and decode have different bottlenecks. Prefill is more attention and matrix-compute heavy. Decode is more sensitive to repeated model-weight loading and KV-cache access.

B. Quantization Integration

The project adds a modular quantization layer through `quantization.py`. The model loading path supports FP16 and 4-bit modes through a shared interface, so quantized and non-quantized runs use the same benchmark logic [5].

This makes comparisons fair because the benchmark code, input generation, and metric logging remain consistent across configurations. The only changes across runs are precision mode and attention backend.

C. Memory and KV-Cache Profiling

The benchmark logs peak GPU memory during both prefill and decode. KV-cache size is computed directly from `past_key_values` by summing key and value tensor sizes across layers:

$$\text{KV bytes} = \sum_l (\text{numel}(K_l) \cdot \text{element_size}(K_l) + \text{numel}(V_l) \cdot \text{element_size}(V_l)) \quad (1)$$

This allows us to separate model parameter memory effects from runtime KV-cache effects.

D. Roofline Analysis

Arithmetic intensity is estimated as:

$$\text{AI} = \frac{\text{FLOPs}}{\text{Bytes moved}} \quad (2)$$

Achieved performance is estimated as:

$$P = \frac{\text{FLOPs}}{\text{Runtime}} \quad (3)$$

The roofline plots are used to interpret whether configurations are memory-bound or compute-bound and how optimizations shift the workload [3].

E. Lightweight End-to-End Benchmark Without Profiler

Before running the profiler-based roofline experiments, we also performed a lightweight end-to-end benchmark without PyTorch profiler tracing. This earlier benchmark used the same four inference configurations and sequence lengths, but avoided profiler overhead. The purpose was to validate that the optimization pipeline worked correctly and to obtain a practical high-level view of latency, throughput, and memory behavior before collecting detailed roofline metrics.

The four configurations were the same as in the main profiling study:

Configuration	Precision	Attention Backend
Baseline	FP16	Standard SDPA / math
FlashAttention	FP16	FlashAttention
Quantized	4-bit	Standard SDPA / math
FA + Quantized	4-bit	FlashAttention

TABLE II
MODEL CONFIGURATIONS

At sequence length 2048, the lightweight non-profiling benchmark showed the same broad trends as the profiler-based study:

Configuration	Prefill Latency	Decode Throughput	Decode Peak Memory
Baseline	1552.55 ms	10.86 tok/s	16194.20 MB
FlashAttention	674.38 ms	15.43 tok/s	16086.80 MB
Quantized	1639.49 ms	12.73 tok/s	6320.70 MB
FA + Quantized	760.75 ms	14.38 tok/s	6217.30 MB

TABLE III
PERFORMANCE COMPARISON ACROSS CONFIGURATIONS

These non-profiling results served as a sanity check for the main experiment. They confirmed that FlashAttention substantially improves practical latency and throughput, while 4-bit quantization significantly reduces memory footprint. The combined FA + Quantized configuration preserved most of the FlashAttention throughput benefit while using memory comparable to the quantized-only configuration.

However, because the goal of the final study is roofline-aware analysis, the main results in Section 4 use the profiler-based measurements. The lightweight benchmark is included only as an early validation step and practical reference point. The profiler-based runs are used for the formal analysis of FLOPs, arithmetic intensity, achieved performance, and roofline placement.

IV. RESULTS

The following results are based on the profiler-enabled benchmark used for roofline analysis. The earlier lightweight benchmark in Section 3.5 showed consistent high-level trends, but the results below are used as the main source for FLOP estimates, arithmetic intensity, achieved performance, and roofline interpretation.

A. Prefill Latency

Prefill latency depends strongly on sequence length and attention backend.

The 128-token point is treated as a warmup / short-context point. It is included in the figures, but the main scaling conclusions are drawn from 256–2048 tokens.

At sequence length 2048:

Configuration	Prefill Latency
Baseline	1668.21 ms
FlashAttention	1043.31 ms
Quantized	1755.49 ms
FA + Quantized	1000.51 ms

TABLE IV
PREFILL LATENCY ACROSS CONFIGURATIONS

At the longest context length, FlashAttention reduces prefill latency substantially. The combined FA + Quantized configuration is the fastest at 2048 tokens, reaching 1000.51 ms compared with 1668.21 ms for baseline.

Quantization alone does not improve prefill latency. At 2048 tokens, quantized-only inference is slightly slower than baseline. This suggests that prefill is primarily limited by attention execution and kernel efficiency rather than model parameter memory alone.

The benefit of FlashAttention is also more meaningful at longer sequence lengths. For short contexts, fixed overheads can dominate, while long contexts provide enough attention work for FlashAttention’s memory-efficient kernel to become beneficial.

Key finding: FlashAttention is the main prefill optimization at long context lengths, while 4-bit quantization alone does not reduce prefill latency.

B. Decode Throughput

Decode throughput shows a different pattern from prefill latency.

At sequence length 2048:

Configuration	Decode Throughput
Baseline	10.74 tok/s
FlashAttention	14.10 tok/s
Quantized	7.15 tok/s
FA + Quantized	7.65 tok/s

TABLE V
DECODE THROUGHPUT ACROSS CONFIGURATIONS

FlashAttention consistently improves decode throughput relative to baseline. At 2048 tokens, FlashAttention improves decode throughput by approximately 31%.

Quantized configurations are slower than FP16 configurations in this implementation. Quantized-only decode reaches 7.15 tok/s at 2048 tokens, and FA + Quantized reaches 7.65 tok/s. This suggests that the memory savings from 4-bit weights are outweighed by dequantization overhead or less optimized quantized kernels during token-by-token generation.

Key finding: FlashAttention improves decode throughput, while 4-bit quantization reduces memory but hurts decode throughput in this setup.

C. Peak GPU Memory

Quantization provides the clearest memory benefit.

At sequence length 2048:

Configuration	Decode Peak Memory
Baseline	16194.18 MB
FlashAttention	16086.84 MB
Quantized	6321.24 MB
FA + Quantized	6217.90 MB

TABLE VI
DECODE PEAK MEMORY ACROSS CONFIGURATIONS

This represents roughly a 61–64% reduction in peak memory for quantized configurations compared with FP16 configurations.

FlashAttention alone does not materially reduce memory footprint. Its primary benefit is performance, not model compression. By contrast, 4-bit quantization substantially reduces both parameter memory and runtime peak GPU memory.

Key finding: 4-bit quantization is mainly a memory-capacity optimization, reducing GPU memory usage from about 16 GB to about 6–7 GB.

D. KV-Cache Scaling

KV-cache memory grows with sequence length and is identical across all configurations:

Sequence Length	KV-cache Memory
128	31.875 MB
256	47.875 MB
512	79.875 MB
1024	143.875 MB
2048	271.875 MB

TABLE VII
KV-CACHE MEMORY VS SEQUENCE LENGTH

The identical KV-cache size across configurations is expected. Weight quantization reduces model parameters, but the KV cache stores runtime key/value activations. Since this project does not quantize the KV cache, KV-cache memory remains unchanged.

This is an important distinction: weight-only quantization reduces static model memory, but it does not solve KV-cache growth for long-context inference. As sequence length, batch size, or number of concurrent requests increases, KV-cache memory can still become a major bottleneck.

Key finding: Weight-only quantization does not reduce KV-cache memory. Long-context serving still requires KV-cache-aware optimization.

E. Arithmetic Intensity

Quantized configurations have much higher arithmetic intensity than FP16 configurations [2], [5].

At sequence length 1024:

Configuration	Arithmetic Intensity
Baseline	1130.53
FlashAttention	1099.69
Quantized	2703.89
FA + Quantized	2658.10

TABLE VIII
ARITHMETIC INTENSITY ACROSS CONFIGURATIONS

At sequence length 2048:

Configuration	Arithmetic Intensity
Baseline	2065.36
FlashAttention	2028.03
Quantized	4695.26
FA + Quantized	4961.61

TABLE IX
ARITHMETIC INTENSITY ACROSS CONFIGURATIONS

Quantization roughly doubles arithmetic intensity because it reduces memory bytes moved per unit computation. FlashAttention alone does not significantly increase arithmetic intensity, but it can improve achieved performance by executing attention more efficiently.

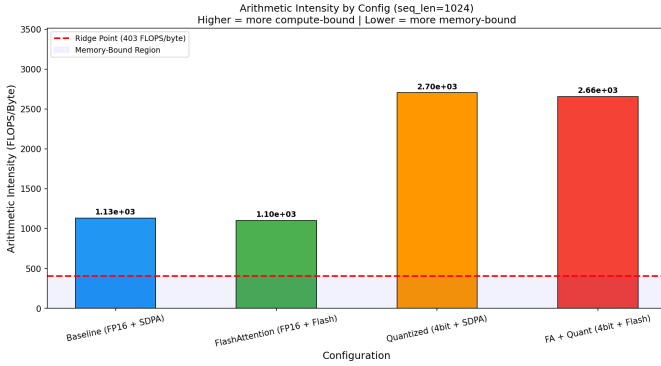


Fig. 1. Arithmetic intensity by configuration at sequence length 1024. Quantized configurations have much higher arithmetic intensity than FP16 configurations because fewer bytes are moved per unit computation. FlashAttention alone does not materially change arithmetic intensity, suggesting that its benefit comes mainly from improved kernel efficiency rather than reduced model-weight memory traffic.

Key finding: Quantization moves roofline points to the right by increasing arithmetic intensity, while FlashAttention mainly improves realized kernel performance.

F. Roofline Interpretation

The roofline plots show two complementary effects.

First, quantization shifts points rightward. By reducing bytes moved for model weights, quantization increases arithmetic intensity. This is visible at both 1024 and 2048 tokens, where quantized configurations have much higher arithmetic intensity than FP16 configurations.

Second, FlashAttention raises achieved performance. FlashAttention does not significantly increase arithmetic intensity by itself, but it improves attention execution efficiency and can move points upward, especially at long sequence lengths.

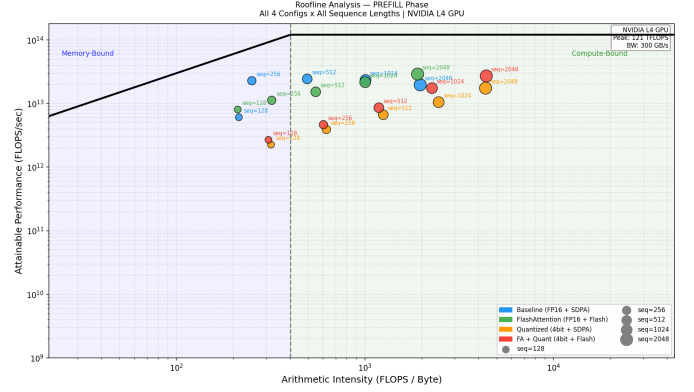


Fig. 2. Roofline analysis for the prefill phase. In prefill, longer sequence lengths generally move points toward higher arithmetic intensity and higher achieved performance. FlashAttention improves achieved performance at longer contexts, while quantized configurations move rightward due to reduced memory traffic. Quantized-only inference has higher arithmetic intensity but does not always achieve higher performance, showing that dequantization and kernel overhead matter.

At sequence length 2048, achieved performance is:

Configuration	Achieved Performance
Baseline	2.10×10^{13} FLOPs/s
FlashAttention	3.13×10^{13} FLOPs/s
Quantized	1.89×10^{13} FLOPs/s
FA + Quantized	3.09×10^{13} FLOPs/s

TABLE X
ACHIEVED PERFORMANCE ACROSS CONFIGURATIONS

This shows why roofline analysis is useful. Quantized-only inference has higher arithmetic intensity than baseline but lower achieved performance. Therefore, higher arithmetic intensity alone does not guarantee higher throughput. Kernel quality, dequantization overhead, memory format conversions, and runtime implementation details also matter.

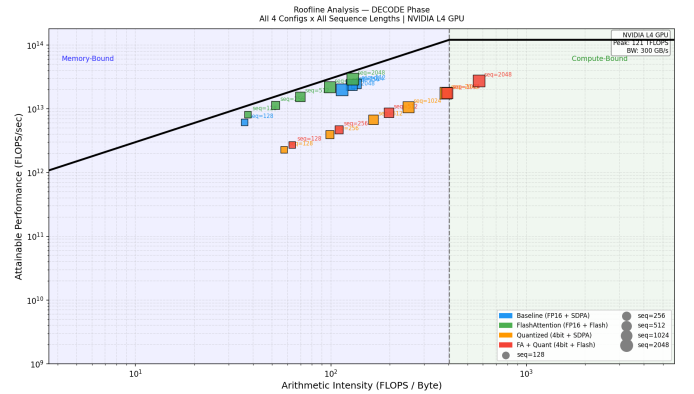


Fig. 3. Roofline analysis for the decode phase. Decode remains more memory-sensitive than prefill because each generated token repeatedly accesses model weights and the KV cache. Quantized configurations move rightward due to reduced weight memory traffic, but their achieved performance is lower than FP16 FlashAttention, indicating that quantization overhead outweighs memory savings in this implementation.

Key finding: Roofline analysis explains why quantization improves memory efficiency but not necessarily speed.

G. System Telemetry

W&B telemetry shows that quantized configurations use much less GPU memory than FP16 configurations. CPU utilization remains comparatively low, and disk/network activity does not dominate the benchmark. This supports the interpretation that observed performance differences are mainly due to GPU compute and memory behavior rather than input pipeline or host-side bottlenecks.

V. DISCUSSION

A. FlashAttention and Quantization Target Different Bottlenecks

FlashAttention targets attention-kernel efficiency. It is most useful when attention memory traffic is large enough to dominate overhead, especially in long-context prefill.

Quantization targets model memory footprint. It substantially reduces GPU memory usage, enabling lower-memory deployment. However, in this experiment, quantization does not improve speed and actually lowers decode throughput.

This distinction is important. FlashAttention should be viewed primarily as a performance optimization, while 4-bit weight quantization should be viewed primarily as a memory-capacity optimization.

B. Quantization Is Not Automatically Faster

The results show that lower precision is not the same as faster inference. Although 4-bit quantization reduces memory footprint and increases arithmetic intensity, the quantized kernels introduce overhead. In token-by-token decode, this overhead can dominate, producing lower throughput than FP16.

This is an important empirical result: quantization should be evaluated as a memory optimization first, and speedup should be treated as workload- and implementation-dependent.

C. KV-Cache Remains a Long-Context Bottleneck

Because KV-cache memory is unchanged by weight-only quantization, long-context inference still faces growing activation memory pressure. At 2048 tokens, KV-cache memory reaches approximately 272 MB. For larger batches, longer contexts, or multi-user serving, KV-cache memory can become a significant bottleneck even when model weights are quantized.

D. Combined Optimization Has Mixed Tradeoffs

FA + Quantized gives strong memory savings and competitive long-context prefill latency, but decode throughput remains below FP16 FlashAttention. This shows that combining optimizations is not always additive.

The best configuration depends on the deployment objective:

TABLE XI
BEST CONFIGURATION BY OPTIMIZATION OBJECTIVE

Objective	Best Choice
Lowest memory usage	Quantized or FA + Quantized
Best decode throughput	FlashAttention
Best long-context prefill latency	FA + Quantized / FlashAttention
Best memory-performance tradeoff	FA + Quantized

E. Role of the 128-token Warmup Point

The 128-token sequence length is included in all figures, but it should be interpreted differently from the main scaling regime. Because it is the first run for each configuration, it may include cold-start effects such as CUDA initialization, memory allocator setup, backend initialization, and GPU clock ramp-up.

Therefore, exact latency values at 128 tokens should not be over-interpreted as steady-state performance. The 128-token point is still useful because it represents a realistic short-context scenario and exposes fixed overheads. However, the main conclusions about scaling, memory behavior, and long-context inference are based primarily on the 256–2048 token range.

VI. LIMITATIONS

- 1) **Single GPU type:** Results are specific to NVIDIA L4.
- 2) **Single model:** We evaluate We evaluate Llama 3.1 8B Instruct.
- 3) **Weight-only quantization:** KV-cache quantization is not implemented.
- 4) **Single-batch inference:** Production serving often uses batching and paged KV cache.
- 5) **128-token warmup effect:** The 128-token point is also the first run and may include cold-start overhead.
- 6) **Estimated roofline:** FLOPs and bytes are estimated using PyTorch profiler and model-based calculations rather than low-level hardware counters from Nsight Compute.
- 7) **Quantized kernel dependency:** 4-bit performance depends heavily on `bitsandbytes` and backend implementation details.

VII. FUTURE WORK

Future work should evaluate:

- KV-cache quantization
- vLLM / PagedAttention
- TensorRT-LLM
- A100 and H100 hardware
- Batch-size scaling
- Multi-user serving workloads
- Longer context lengths
- Nsight Compute hardware-counter roofline analysis
- Tokens-per-watt efficiency

VIII. CONCLUSION

This project shows that LLM inference optimization must be phase-aware and bottleneck-aware. FlashAttention improves attention execution and provides the strongest throughput benefit, especially for decode and long-context prefill. 4-bit weight quantization significantly reduces GPU memory footprint and increases arithmetic intensity, but in our implementation it does not improve speed and reduces decode throughput due to likely dequantization and kernel overhead.

KV-cache memory grows with sequence length and remains unchanged across all configurations, confirming that weight-only quantization does not compress runtime key/value activations. The 128-token point is reported as a warmup / short-context case, while the main scaling conclusions are drawn from the 256–2048 token range.

The strongest conclusion is: FlashAttention is primarily a performance optimization, while 4-bit weight quantization is primarily a memory-capacity optimization. Their combination provides a useful memory-performance tradeoff, but the gains are not perfectly additive.

This conclusion is supported by latency, throughput, memory profiling, KV-cache tracking, and roofline analysis.

REFERENCES

- [1] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, “FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness,” *arXiv preprint arXiv:2205.14135*, 2022.
- [2] J. Lang, Z. Guo, and S. Huang, “A Comprehensive Study on Quantization Techniques for Large Language Models,” *arXiv preprint arXiv:2411.02530*, 2024.
- [3] S. Williams, A. Waterman, and D. Patterson, “Roofline: An Insightful Visual Performance Model for Multicore Architectures,” *Communications of the ACM*, vol. 52, no. 4, pp. 65–76, 2009.
- [4] Meta, “Llama 3.1 8B Instruct Model Card,” *Hugging Face*, 2024.
- [5] Hugging Face, “Bitsandbytes Quantization,” *Transformers Documentation*.
- [6] PyTorch, “torch.nn.functional.scaled_dot_product_attention,” *PyTorch Documentation*.

AI USAGE DISCLOSURE

Did you use any AI tool in completing this assignment?

Yes, we used AI assistance as described below.

Tool(s) used: ChatGPT, Claude

Specific purpose: Debugged implementation issues, clarified GPU roofline concepts, assisted with code structure and visualization, polished writing and formatting.

Questions/sections affected: Infrastructure setup, profiling implementation, roofline plot generation, W&B dashboard.

How you verified correctness: Ran all code on GCP, confirmed outputs matched expected results, manually verified calculation formulas against hardware specs.